

COMPUTER VISION

Unit 1: Image Formation & Fundamentals

Unit Overview

Unit No.	Topic Name	Sub-Topics Covered	Hours	CO Code
Unit 1	Image Formation & Fundamentals	Intro to CV & Visual Learning, Human vs Machine Vision; Image formation & pinhole camera model; Sampling & quantization; Colour models & image representation; Basic image processing (filtering & enhancement)	09	CO1

Introduction to Computer Vision and Visual Learning

What is Computer Vision?

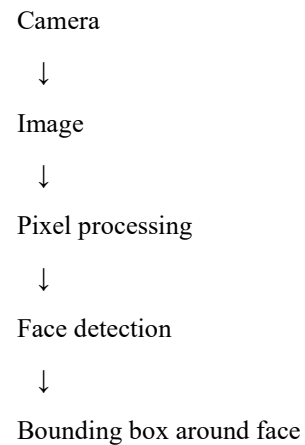
Computer Vision (CV) is the field of study that enables computers to derive meaningful, high-level understanding useful information from digital images or videos, in the same way that biological vision allows humans and animals to interpret their surroundings.

In simple terms:

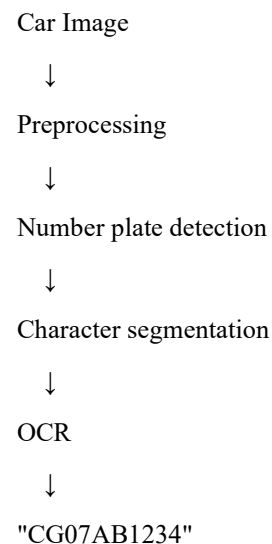
Image → Computer processes pixels → Extracts information → Makes a decision

Formally, Computer Vision can be defined as the automatic extraction, analysis, and understanding of useful information from a single image or a sequence of images.

For example: **Face detection**



Number plate recognition



What is an image?

An image is essentially a 2D array of numerical values.

For example, a grayscale image might look like:

```
10 20 30 40
15 25 35 45
20 30 40 50
25 35 45 55
```

Each number represents the intensity/brightness of one location.

- *So, Image is collection of pixels where each pixel contains numerical information.*
 - *A computer works with these numbers.*
 - *A 1920×1080 image contains:*

$$1920 \times 1080 = 2,073,600 \text{ pixels.}$$
So approximately 2 megapixels.
 - *Pixel = Picture Element*
 - *A pixel is the smallest individual element of a digital image.*
-

Computer Vision vs Image Processing

These two concepts are closely related but are **not the same**.

Image Processing

The goal is generally to **modify or improve an image**.

Examples:

- Resize image
- Remove noise
- Increase brightness
- Sharpen image
- Convert RGB $\rightarrow$ grayscale
- Blur image
- Enhance contrast

Input Image



Image Processing



Improved Image

Computer Vision

The goal is to **understand what is present in an image**.

Examples:

- Is there a person?
- Is this a cat or dog?
- Where is the car?
- How many people are present?
- Is the person wearing a helmet?

Input Image → Computer Vision → Understanding / Decision

Important relationship

Image Processing → Preprocessing → Computer Vision → Prediction / Understanding

So, image processing is often a **preprocessing stage of a CV system**.

What is Visual Learning?

Visual Learning refers to the process by which a system (biological or artificial) acquires the ability to interpret visual data through experience, i.e., by learning statistical regularities and patterns from a large number of example images rather than being explicitly programmed with hand-crafted rules.

In simple words, Visual learning refers to machine learning/deep learning systems that learn patterns from visual data.

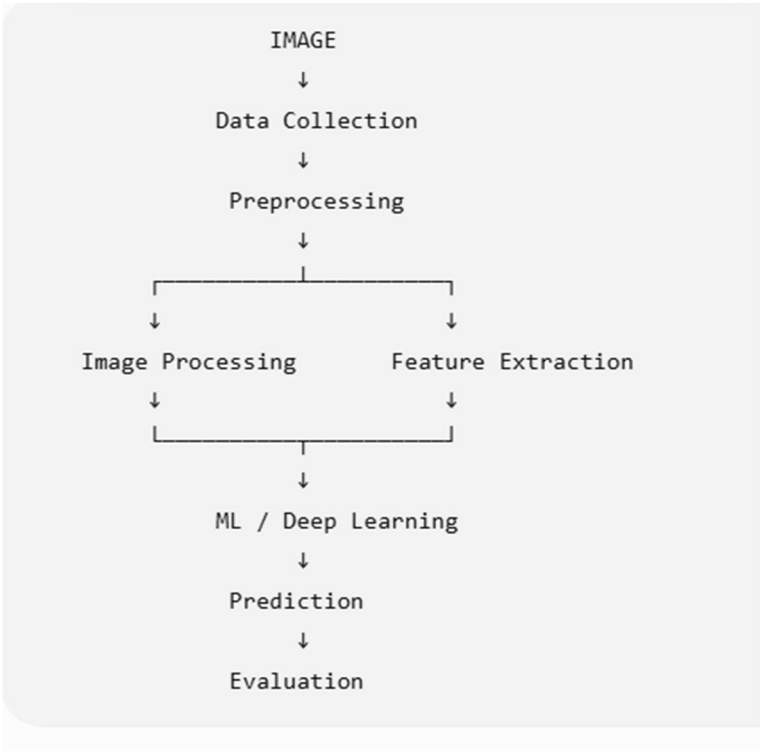
Examples:

Task	Input	Output
Image Classification	Image	Cat
Object Detection	Image	Cat + location
Image Segmentation	Image	Pixel-level regions
Face Recognition	Face image	Person identity
OCR	Text image	Text

Goals / Applications of Computer Vision

Application Area	Typical Tasks	Examples
Object Recognition	Detection, classification, localization	Face recognition, pedestrian detection
Scene Understanding	Segmentation, depth estimation, 3-D reconstruction	Autonomous driving perception stack
Motion Analysis	Optical flow, tracking, action recognition	Sports analytics, surveillance
Medical Imaging	Segmentation, anomaly detection	Tumor detection in MRI/CT scans
Industrial	Quality inspection, robotics guidance	PCB defect detection, bin picking
Biometrics	Iris/fingerprint/face recognition	Airport security, phone unlock
AR/VR	Pose estimation, SLAM	Markerless AR overlays

A practical CV pipeline often looks like:



Human Vision vs Machine Vision

Human Vision System

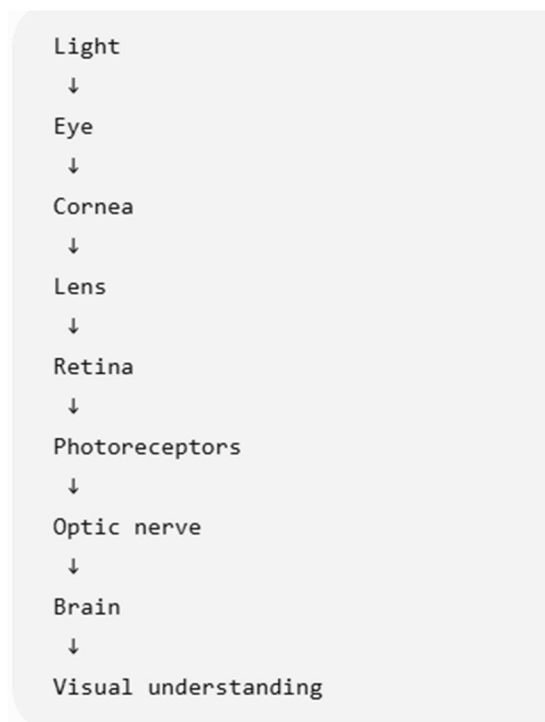
The human visual system begins with light entering the eye through the cornea and lens, which focuses an image onto the retina.

The retina contains two types of photoreceptors: rods (about 120 million, responsible for vision in low light and motion detection, but not color) and cones (about 6 million, responsible for color vision and fine detail, concentrated in the fovea).

Photoreceptor signals are pre-processed by retinal ganglion cells and transmitted via the optic nerve to the Lateral Geniculate Nucleus (LGN) and then to the primary visual cortex (V1) in the occipital lobe, where edge and orientation detection occurs.

Higher visual areas (V2, V4, IT cortex) progressively extract more complex features, ultimately enabling object recognition in the inferotemporal cortex.

Simplified Process



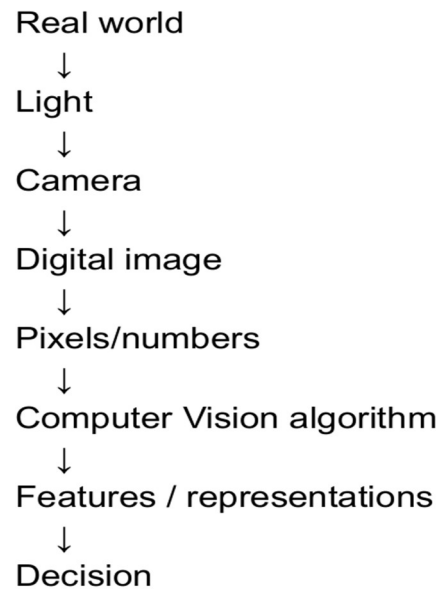
The retina contains:

- **Rods** → low-light vision
- **Cones** → color vision

The brain then interprets the information.

Machine Vision

A computer vision system uses:



Seeing a picture in which a red color car is parked.

A human may immediately understand: "There is a red car parked beside a road."

A computer initially sees something like:

Pixel(0,0) = [123, 80, 54]

Pixel(0,1) = [125, 82, 55]

Pixel(0,2) = [130, 85, 60]

...

It doesn't inherently know that these pixels represent a car.

A CV model must **learn visual patterns**.

Human Vision vs Machine (Computer) Vision

Human Vision

Eyes capture light

Retina processes signals

Brain interprets

Very strong contextual understanding

Learns naturally

Highly adaptable

Excellent generalization

Machine Vision

Camera captures light

Sensor produces digital values

Algorithms/models interpret

Context can be difficult

Requires data/training

Can fail under distribution changes

Usually task-dependent

Levels of Processing in a Computer Vision System (Marr's Paradigm)

David Marr proposed a very influential three-level framework for understanding vision, which is often used to structure CV pipelines:

1. Low-level vision: pixel-level operations such as edge detection, filtering, noise removal, and computation of the 'primal sketch' (basic image features like edges and blobs).
2. Mid-level vision: grouping and organization of low-level features into the '2.5-D sketch' — surfaces, depth, and orientation relative to the viewer, using cues like stereo, shading and texture.
3. High-level vision: object recognition, scene interpretation and construction of a full 3-D model of the scene, independent of viewpoint.

Some Important Questions & Answers

1. Define Computer Vision. How does it differ from Image Processing?
 - Image Processing focuses on transforming an image into another image (image-to-image, e.g., enhancement, denoising) whereas Computer Vision focuses on extracting meaning/understanding from an image (image-to-information, e.g., recognizing an object). Image processing is often a pre-processing stage used within a larger CV pipeline.
2. Differentiate between rods and cones in the human eye and explain their role in vision.
 - Rods (~120 million) are highly sensitive to light, enable scotopic (low-light) and peripheral vision, but do not detect color.
 - Cones (~6 million) are concentrated in the fovea, function in bright light (photopic vision), and provide color vision and high spatial acuity via three types sensitive to short, medium and long wavelengths.

Practical / Lab Exercise

Aim

To install and set up a Python-based Computer Vision environment (OpenCV, NumPy, Matplotlib), read a digital image, display its basic properties, and view it — as an introductory exercise to appreciate how a 'scene' becomes a numeric array a machine can process.

Requirements

- Python 3.8+, pip
- Libraries: opencv-python, numpy, matplotlib

Setup

```
pip install opencv-python numpy matplotlib
```

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read an image from disk (BGR order by default in OpenCV)
img = cv2.imread('sample.jpg')

if img is None:
    raise FileNotFoundError('Check the image path')

# Basic properties -> demonstrates that an image is just a numeric array
print('Image shape (H, W, Channels):', img.shape)
print('Data type:', img.dtype)
print('Total pixels:', img.shape[0] * img.shape[1])
print('Pixel value at (0,0):', img[0, 0])

# Convert BGR (OpenCV default) to RGB for correct display with matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('Loaded Image')
plt.axis('off')
plt.show()
```

Expected Output

The console prints the image's shape (e.g., (408, 612, 3) meaning 408 rows, 612 columns, 3 colour channels), its data type (uint8), the total pixel count: 249696, and the BGR value of the top-left pixel. The Matplotlib window displays the loaded photograph in correct colours.

Image Formation Process and the Pinhole Camera Model

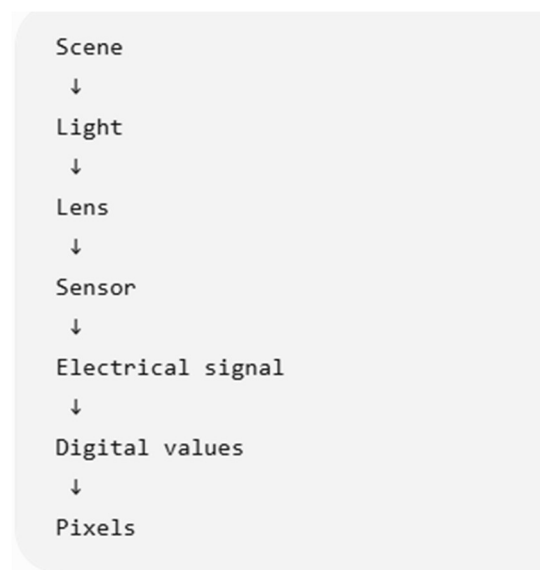
The Image Formation Process

Image formation is the physical process by which

Light reflected or emitted from objects in a 3-D scene enters the camera lens

The lens focuses the light onto the image sensor.

The sensor converts the light into electrical signals, which are then converted into digital values
(a 2-D array of intensity/colour values (an image)).



Three main factors govern image formation:

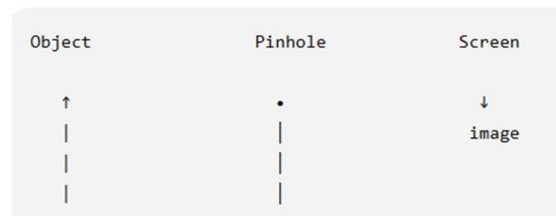
- Geometry: how 3-D points project onto the 2-D image plane (perspective projection), determined by camera position, orientation and optics.
- Radiometry (Photometry): how much light reaches the sensor from a given scene point, determined by illumination, surface reflectance (BRDF), and sensor sensitivity.
- Sensing: how continuous light intensity is converted into discrete digital numbers by the camera sensor (sampling + quantization + noise).

The Pinhole Camera Model

- The pinhole camera is the simplest and most fundamental model of image formation.
- It consists of a light-tight box with a very small aperture (the 'pinhole') on one side, through which light rays from the scene pass and form an inverted image on the opposite side (the image plane).
- Because the aperture is infinitesimally small, only a single ray of light from each scene point can pass through it, resulting in a sharp, focused image without the need for a lens (in the idealized model).

Imagine a completely dark box.

Put a tiny hole on one side.



Light rays from the object pass through the tiny hole and create an image on the opposite side.

The resulting image is inverted.

Geometry of Perspective Projection

Let the pinhole (also called the centre of projection or optical centre) be located at the origin O , and let the optical axis be the z -axis. The image plane is placed at a distance f (the focal length) from the pinhole, along the optical axis.

Consider a 3-D scene point $P = (X, Y, Z)$ expressed in the camera coordinate system.

$$x = f \frac{X}{Z}$$

$$y = f \frac{Y}{Z}$$

The important concept is:

Objects farther away appear smaller.

This relationship is called the perspective projection equation.

Homogeneous Coordinates and the Projection Matrix

In Computer Vision, we commonly write the projection as:

$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

where K is the **camera intrinsic matrix**.

Typically:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

You don't need to deeply memorize this initially.

Understand the idea:

A camera converts 3D world coordinates into 2D image coordinates.

Practical / Lab Exercise

Aim

To simulate perspective projection of 3-D points using the pinhole camera model in Python, and to perform basic camera calibration to estimate intrinsic parameters and correct lens distortion using OpenCV.

Simulating Perspective Projection

```
import numpy as np

# Camera intrinsic matrix (fx, fy, cx, cy in pixels)
K = np.array([[800, 0, 320],
              [0, 800, 240],
              [0, 0, 1]], dtype=float)

# A set of 3-D points in the camera coordinate frame (X, Y, Z) in metres
points_3d = np.array([
    [0.30, 0.15, 3.0],
    [-0.20, 0.10, 2.0],
    [0.00, 0.00, 1.0],
])

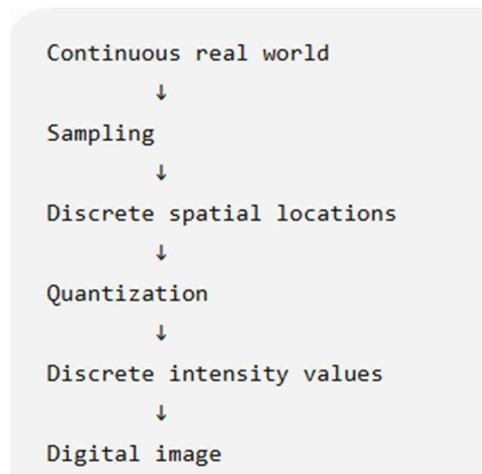
def project(points_3d, K):
    # Convert to homogeneous image coords: s*[x,y,1]^T = K * [X,Y,Z]^T
    proj = (K @ points_3d.T).T
```

```
# Perspective divide by Z (the third component before normalisation)
pixels = proj[:, :2] / points_3d[:, 2:3]
return pixels

pixel_coords = project(points_3d, K)
print('Projected pixel coordinates:\n', pixel_coords)
```

Sampling and Quantization

- A real-world scene has
 - continuous spatial variation (i.e. domain of the image is continuous $(x, y \in \mathbb{R})$ and
 - continuous intensity variation — light intensity is a continuous function $f(x, y)$ of continuous spatial coordinates.
- Light varies infinitely across space and brightness transitions smoothly from deep shadows to light shades and to store and process this on a digital computer, the continuous signal must be converted into a discrete, finite representation through two independent operations:
- sampling (discretizing the spatial coordinates) and
- quantization (discretizing the intensity/amplitude values)



Sampling

- Sampling = deciding WHERE we measure the image.
- Sampling is the process of measuring the value of the continuous image function $f(x, y)$ at a discrete, evenly spaced grid of spatial locations, producing an $M \times N$ array of samples (pixels).
- In an image, sampling determines the **spatial resolution**.
- For example:
 - 500×500
 - 1920×1080
 - 3840×2160
- More samples generally mean higher spatial detail.
- If sampling is too low, important details can disappear.
- The spacing between samples determines the spatial resolution of the digital image:
 - A smaller sampling interval (higher sampling rate) captures finer spatial detail, while
 - A larger interval loses detail and may introduce aliasing.
- **Sampling Interval** is physical distance between the centres of two adjacent sensor pixels (often called **pixel pitch**).
- **Sampling Rate** (f_s): The reciprocal of the interval. It measures how many samples (pixels) are collected per unit of physical distance (e.g., pixels per millimeter or DPI).

possible intensity values.

Examples:

0 → black
 50 → dark gray
 128 → medium gray
 200 → light gray
 255 → white

4-bit Image

If we use only 4 bits:

$$2^4 = 16$$

levels.

So instead of 256 intensity levels: 0, 1, 2, ..., 255

we have: 0, 1, 2, ..., 15

This produces less intensity precision.

Sampling vs Quantization

Think:

Sampling

↓

How many pixels?

Quantization

↓

How many brightness levels per pixel?

Image Representation as a Matrix

After sampling and quantization, a digital image is completely represented as a 2-D matrix

For grayscale: $f(x, y)$, $x = 0 \dots M-1$, $y = 0 \dots N-1$, where each element is an integer intensity value in $[0, L-1]$.

A colour image is represented as a 3-D array ($M \times N \times 3$) with separate matrices for each colour channel (commonly Red, Green, Blue).

Program: Effect of Quantization (Bit-depth Reduction)

```
import cv2, numpy as np
import matplotlib.pyplot as plt

img = cv2.cvtColor(cv2.imread('sample.jpg'), cv2.COLOR_BGR2GRAY)

def quantize(img, levels):
    step = 256 // levels
    return (img // step) * step

bit_levels = [256, 16, 8, 4, 2] # corresponds to 8,4,3,2,1 bits
fig, axes = plt.subplots(1, len(bit_levels), figsize=(15, 4))

for ax, L in zip(axes, bit_levels):
    q = quantize(img, L)
    ax.imshow(q, cmap='gray', vmin=0, vmax=255)
    ax.set_title(f'{L} levels')
    ax.axis('off')

plt.tight_layout()
plt.show()
```